

SEBA Task A03

G01 - T13

Task a

Categorization

(The summary table is on page 3)

1. **/LF10/ Registration:** The user must be able to register on the application.
 - Input: user enters the required data to register
 - Query: the data from the new user is written into the database
2. **/LF20/ Document upload:** A logged-in user must be able to upload the required documents: ID card, driver's licenses.
 - Input: user uploads the documents
3. **/LF30/ Save payment methods:** A logged-in user must be able to store his most preferred payment types on the application.
 - Input: user chooses the preferred payment type
4. **/LF40/ Update account details:** A logged-in user must be able to update the account details such as email address and password.
 - Input: user enters the new data
 - Query: updating data in the database
5. **/LF50/ Delete account:** The logged-in user must be able to delete his account on the application.
 - Input: user commands for the account to be removed
 - Query: the user is removed from the database
6. **/LF60/ Log-in:** A user must log in to the system with his email address and password before he can reserve and book vehicles.
 - Input: user enters the email & password
 - Query: the credentials are being checked with the existing user data
7. **/LF70/ Log-out:** A logged-in user can log out of the system after he has completed his activities.
 - Input: the user requested to log out (user action)
8. **/LF80/ Interactive map:** The application must show an interactive map of Munich with all available vehicles.

- Query: the system sorts out all vehicle available
 - Output: the app displays the map
9. **/LF90/ Filtering:** The user must be able to filter for vehicles based on the vehicle type and properties.
- Input: the user picks the options for the filter
10. **/LF100/ Vehicle properties:** The application must show the properties of a vehicle when the user clicks on it on the map.
- Query: accessing the properties vehicle
 - Output: the app shows the result
11. **/LF110/ Check driver's license:** The application must check for a valid driver's license of the logged-in user when he chooses to rent a car.
- Output: allowing or disallowing the user to rent a car
12. **/LF120/ Enquire payment method:** The application must ask for the logged-in user's preferred type of payment from the listed choices upon his request for a rental.
- Output: the app asks for the type of payment
13. **/LF130/ Reserve vehicle:** A logged-in user can reserve a vehicle for 10 minutes.
- Input: the user reserves a vehicle
 - Query: the status of the vehicle is updated
14. **/LF140/ Users cancels reservation:** A logged-in user can cancel the reservation.
- Input: the user cancels the reservation
 - Query: the status of the vehicle is updated
15. **/LF150/ System cancels reservation:** A reservation by a customer expires automatically after 10 minutes if the customer has not rented the vehicle yet.
- Output: the system cancels the reservation
16. **/LF160/ Start rental:** A logged-in user must be able to start and end a rental by entering a 7- digit code (generated by the system) at the vehicle.
- Input: user enters the code, user starts & ends the rental
17. **/LF170/ Start rental confirmation:** After the rental was started, the system must display a confirmation that the vehicle was rented successfully.
- Output: the app displays the confirmation
18. **/LF180/ Trip status:** A logged-in user must be able to view the trip's current duration and fee.
- Output: the system gives out the duration & fee
19. **/LF190/ End rental:** A logged-in user must be able to end a rental at a service point.

- Input: the user ends the rental
20. **/LF200/ End rental confirmation:** After returning the bicycles at a service point, the system must display the total price and a confirmation that the vehicle was returned successfully.
- Output: the system gives out the price & confirmation
21. **/LF210/ Display simple usage statistics:** A logged-in user can see statistics regarding their usage (all past rentals with the corresponding durations and costs).
- Query: accessing the database for the user statistics
 - Output: displaying the result
22. **/LF220/ Support/Emergency contact:** The application should provide the contact details of a support/emergency contact.
- Query: accessing the database for the contact
23. **/LD10/: Customer data** (1 record element type, <50 data element types)
- Reference Data: belongs to the customer (external party)
24. **/LD20/: Vehicle data** (1 record element type, <50 data element types)
- Database: vehicles belong to the company
25. **/LD30/: Rental data** (1 record element type, <50 data element types)
- Database: saved by the company
26. **/LD40/: Payment data** (1 record element type, <50 data element types)
- Database: stored in the company database
27. **/LD50/: Map data** (1 record element type, <50 data element types)
- Reference Data: dependant on third party app e.g. google maps

Input	/LF10/, /LF20/, /LF30/, /LF40/, /LF50/, /LF60/, /LF70/, /LF90/, /LF130/, /LF140/, /LF160/, /LF190/
Output	/LF80/, /LF100/, /LF110/, /LF120/, /LF150/, /LF170/, /LF180/, /LF200/, /LF210/
Query	/LF10/, /LF40/, /LF50/, /LF60/, /LF80/, /LF100/, /LF130/, /LF140/, /LF210/, /LF220/
Database	/LD20/, /LD30/, /LD40/
Reference Data	/LD10/, /LD50/

Classification

	Simple	Medium	Complex
Input	/LF10/, /LF20/, /LF30/, /LF40/, /LF50/, /LF60/, /LF70/, /LF90/, /LF140/, /LF160/, /LF190/	/LF130/	
Output	/LF100/, /LF110/, /LF120/, /LF150/, /LF170/, /LF180/, /LF200/, /LF220/	/LF80/, /LF210/	
Query	/LF10/, /LF40/, /LF50/, /LF60/, /LF100/, /LF140/	/LF80/, /LF130/, /LF210/	
Database	/LD20/, /LD30/, /LD40/		
Reference Data	/LD10/, /LD50/		

Weighting of the Influencing Factors

1	Integration with other application systems (0-5)	5	Dependencies with external system (map, reservation, payment)
2	Decentralizing data, decentralized processing (0-5)	4	Quite decentralized (customer database, payment information)
3	Transaction rate (0-5)	5	There could be a lot transactions or reservation during rush hour
4	Processing logic: a) Calculations (0-10) b) Input Validation (0-5) c) Exception handling (0-10) d) Logic (0-5)	5 5 10 3	Has to process simple calculation such as rent duration, fee Need to confirm bank / payment informations & driver's license Many possible errors, wrong payment data, invalid license, wrong confirmation, wrong fee, wrong vehicle type Demanding simple logic for user, transaction / rent
5	Reusability (0-5)	0	No Information
6	Database migration (0-5)	3	The payment and map data need to be integrated

7	Customizability (0-5)	5	A long term goal of the management board is to add another type of vehicle such as scooters
----------	-----------------------	----------	---

Calculation Form

Including: Evaluation of the Influence Factors & Calculation of the Evaluated Function Points

Category	Quantity	Classification	Weighting	Row sum
Input data	11	simple	x 3	33
	1	medium	x 4	4
	-	complex	x 6	-
Queries	6	simple	x 3	18
	3	medium	x 4	12
	-	complex	x 6	-
Outputs	8	simple	x 4	32
	2	medium	x 5	10
	-	complex	x 7	
Databases	3	simple	x 7	21
	-	medium	x 10	-
	-	complex	x 15	-
Reference data	2	simple	x 5	10
	-	medium	x 7	-
	-	complex	x 10	-
Sum (E1)				140
Influencing Factors (change the Function-Point-Value by ± 30%)		1. Integration with other application systems (0-5)	5	
		2. Decentralized data, decentralized processing (0-5)	4	
		3. Transaction rate (0-5)	5	

	4. Processing logic:	
	a) Calculations (0-10)	5
	b) Input validation (0-5)	5
	c) Exception handling (0-10)	10
	d) Logic (0-5)	3
	5. Reusability (0-5)	0
	6. Database migration (0-5)	3
	7. Customizability (0-5)	5
Sum of 7 influences (E2)		45
Influence Factor Weighting = $E2 / 100 + 0.7 = E3$		1.15
Calculated Function Points: $\text{round}(E1 * E3)$		161

Determination of the Effort in Person Months

Using the Function Point - Value pairs table according to IBM 85 [Balzert 2001], the necessary man-months (MM) is 14 MM.

Task b

Evaluation

14 MM is too low to develop this relatively big app. For example if we convert 14 MM into 7 people working in 2 months, in this case the team doesn't have a lot of time to develop, evaluate and test. For this kind of app, it will probably need 3-4 months in a team of 7 people. This means that 21-28 MM would be needed.

There are many advantages that prove to be true while we were implementing the Function Point Method in this task.

- Easy to learn
- Well-defined procedural steps

The steps are well-planned and rather straight-forward without new concepts needed to be learnt. Thus making this method even accessible to beginners.

- Low time requirement
- Tool support available
- First estimate possible at very early stage

The accessibility of this method and availability of tool support leads to low conducting time. Thus bringing about an early estimate for the software development stage.

- Adaptability to new techniques
- Adaptability to different application areas
- Adaptability to company-specific environment

Moreover, the Function Point Method still stands as a prominent method, even in the agile world as it's open to changes to fit into specific cases.

However, there are two disadvantages that we can use as arguments for our team's evaluation:

- Too strongly function-oriented
- Subjective judgement

There is more to just catering for functions when one develops an application. Furthermore, because of the subjective judgement, especially around Weighting influencing factors, it has led us to question the end result of the development time. Having already taken Sneed's Devil Square into account, we still find there was a one-month deviation between the calculated estimate (using FPM) and our overall estimate of the development time for this mid-size app.